

Worksheet 5 — Cross-Site Scripting & Client-Side Risks (3 hrs)

Course: Software Security (KOSEN69) · **Week 5 Aligned:** OWASP 2025 **A05 Injection** · **CWE-79** (XSS), **CWE-352** (CSRF), **CWE-1004** (cookie without HttpOnly) **Signature game:** 🚩 **XSS Golf** — fire `alert(1)` in the fewest characters possible. Lower payload length = lower score = better. Par for reflected is the `<img>` vector; can you go under par?

⚠️ **Ethics note:** Use only the provided `vulnerable_app.py` sandbox and your own Juice Shop container. Stealing real users' cookies or sessions is illegal. All "session theft" steps here target the sandbox cookie `session=abc123` only.

Part 1 — Student Information

Name	Student ID	Date	Group
Sai Shang Hlang	6631503129	15 Sep	-

AI-use disclosure: I used an AI coding assistant (Claude) to stand up and verify the Week 5 lab (Tasks 1–5 against both `vulnerable_app.py` and `fixed_app.py`) and to help draft the worksheet answers, including the Audit-the-AI example and the Prompt-Problem final prompt. The exploit/defense evidence (unescaped payloads, escaped output, CSP header, cookie flags, CSRF replay) was produced and verified by actually running the lab on my machine.

Part 2 — Lecture Questions

Answer in 2–4 sentences each.

1. Distinguish **reflected**, **stored**, and **DOM-based** XSS by *where* the untrusted data is injected and *when* it executes. Which two does our `vulnerable_app.py` implement, and at which routes?
 - Reflected XSS injects untrusted data that comes from the current request (e.g. a URL query parameter) straight into the response, and it executes only in that one request/response cycle (e.g. `/hello?name=`). Stored XSS persists the attacker's input on the server (e.g. a comment) and re-serves it to every future visitor, executing whenever any victim loads the page (e.g. `/comments`). DOM-based XSS never reaches the server with the payload — the server sends trusted HTML, and vulnerable client-side JavaScript reads a source (location, fragment, etc.) and writes it into the DOM unsafely. Our `vulnerable_app.py` implements **reflected** XSS at `/hello` (L17–20) and **stored** XSS at `/comments` (L26–30); it does not expose a DOM-based sink.
2. How does **contextual output encoding** (`markupsafe.escape`) stop `<script>` from executing? Why is HTML-context encoding different from JavaScript- or URL-context encoding?
 - `escape()` turns the metacharacters `<`, `>`, `&`, and quotes into HTML entities (`<`, `>`, `&`, `"`), so the browser parses the input as *text* inside the page rather than as a new tag or attribute. It is "contextual" because the safe encoding differs by sink: inside an attribute you must also neutralize the quote that closes the attribute; inside a JavaScript string you must prevent the string from being closed (e.g. backslashes and line breaks);

inside a URL you must prevent scheme/parameter injection. Escaping for the wrong context leaves a bypass.

3. Explain how a strict **Content-Security-Policy** (`script-src 'self'`) defeats an *injected* inline script even when encoding is missing.
 - A strict CSP tells the browser which scripts it is allowed to load and execute. With `script-src 'self'`, the browser will only run scripts from the same origin and rejects inline `<script>`, inline event handlers like `onerror`, and `javascript:` URLs. So even if encoding is missing and a `<script>` tag is injected into the HTML, the browser refuses to execute it (and logs a CSP violation in DevTools), which makes CSP a strong defense-in-depth layer behind output encoding.
4. What do the cookie flags **HttpOnly**, **SameSite**, and **Secure** each protect against? Map each to a concrete attack (cookie theft via XSS, CSRF, network sniffing).
 - **HttpOnly** stops client-side JavaScript from reading the cookie via `document.cookie`, so an XSS payload cannot steal the session cookie (blocks **cookie theft via XSS**, CWE-1004). **SameSite** controls whether the cookie is attached to cross-site requests, so `SameSite=Strict/Lax` stops the browser from sending `session` on a forged cross-site request (blunts **CSRF**). **Secure** ensures the cookie is only transmitted over HTTPS, protecting it from **network sniffing** on plaintext connections.
5. Why does **CSRF** (CWE-352) work even without any script injection, and how does `SameSite=Strict` plus the same-origin policy blunt it?
 - CSRF needs no script because the *browser itself* automatically attaches applicable cookies to a request. A forged form on an attacker's page that POSTs to a state-changing endpoint therefore carries the victim's `session` cookie cross-site, so the server treats it as the victim acting. `SameSite=Strict` stops the browser from attaching the cookie on cross-site requests, and the same-origin policy stops the attacker's page from *reading* any cross-site response. But the same-origin policy does not stop the request from being *sent*, so a server-side CSRF token tied to the session is still the authoritative control.

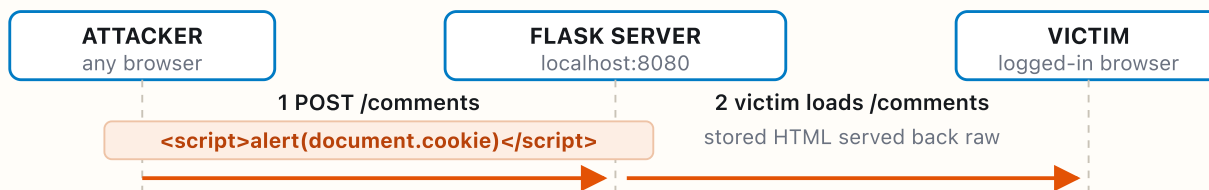
Part 3 — Hands-on Lab (150 min)

Same browser, same cookie, opposite directions

1. STORED XSS (cross-site scripting) · CWE-79

attacker → server → victim

Written once by the attacker, run in every visitor's browser.



3. The script runs in the VICTIM's origin, not the attacker's.

No HttpOnly (CWE-1004) → document.cookie leaks session=abc123

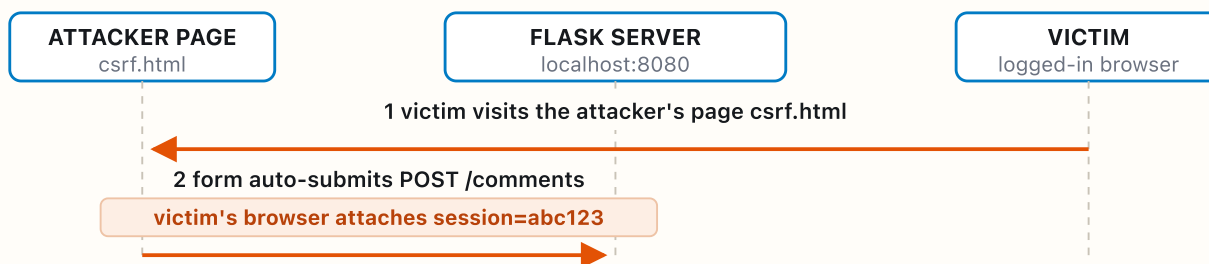
FIX — escape() on /hello, autoescape on /comments,
plus Set-Cookie: HttpOnly (fixed_app.py L21, L30-33, L42)

✗ SameSite does nothing
payload is same-origin

2. CSRF (cross-site request forgery) · CWE-352

victim → attacker page → server

No script injected. The victim's own browser sends the request.



3. The server sees a normal, authenticated request.

It cannot tell the victim never meant to send it — CWE-352

FIX — SameSite=Strict stops the browser attaching it,
and a CSRF token the server actually checks on POST.

✗ HttpOnly does nothing
no script reads it

- fixed_app.py adds encoding, CSP and HttpOnly — the XSS dies.
- The CSRF still works: /comments checks no token on POST.

Learning goals: land reflected + stored XSS, abuse a JS-readable cookie, build a CSRF PoC against the comment board, then prove `fixed_app.py` blocks all of it.

Prerequisites: Docker + Docker Compose, a browser with DevTools, a text editor. Working dir: `labs/week05-xss-client-side/`.

Environment setup

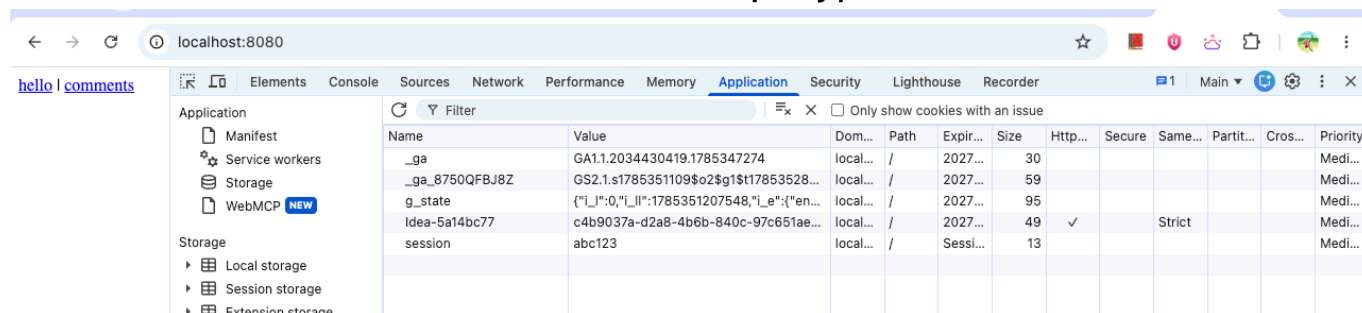
```
cd labs/week05-xss-client-side
docker compose up          # python:3.12-slim + flask, runs
vulnerable_app.py
# vulnerable app -> http://localhost:8080 (service name: xss-lab, port
8080)
```

Optional secondary target (for DOM XSS, which our app does not expose):

```
docker run --rm -p 3000:3000 bkimminich/juice-shop # ->
http://localhost:3000
```

What to submit per task: the exact **payload**, a **screenshot** of the alert/effect, and a **2–3 sentence mitigation**.

Task 0 — Onboarding (5 min). Browse <http://localhost:8080/>. Open DevTools → Application → Cookies and confirm **session=abc123** is set with **no HttpOnly / SameSite**. Screenshot it.

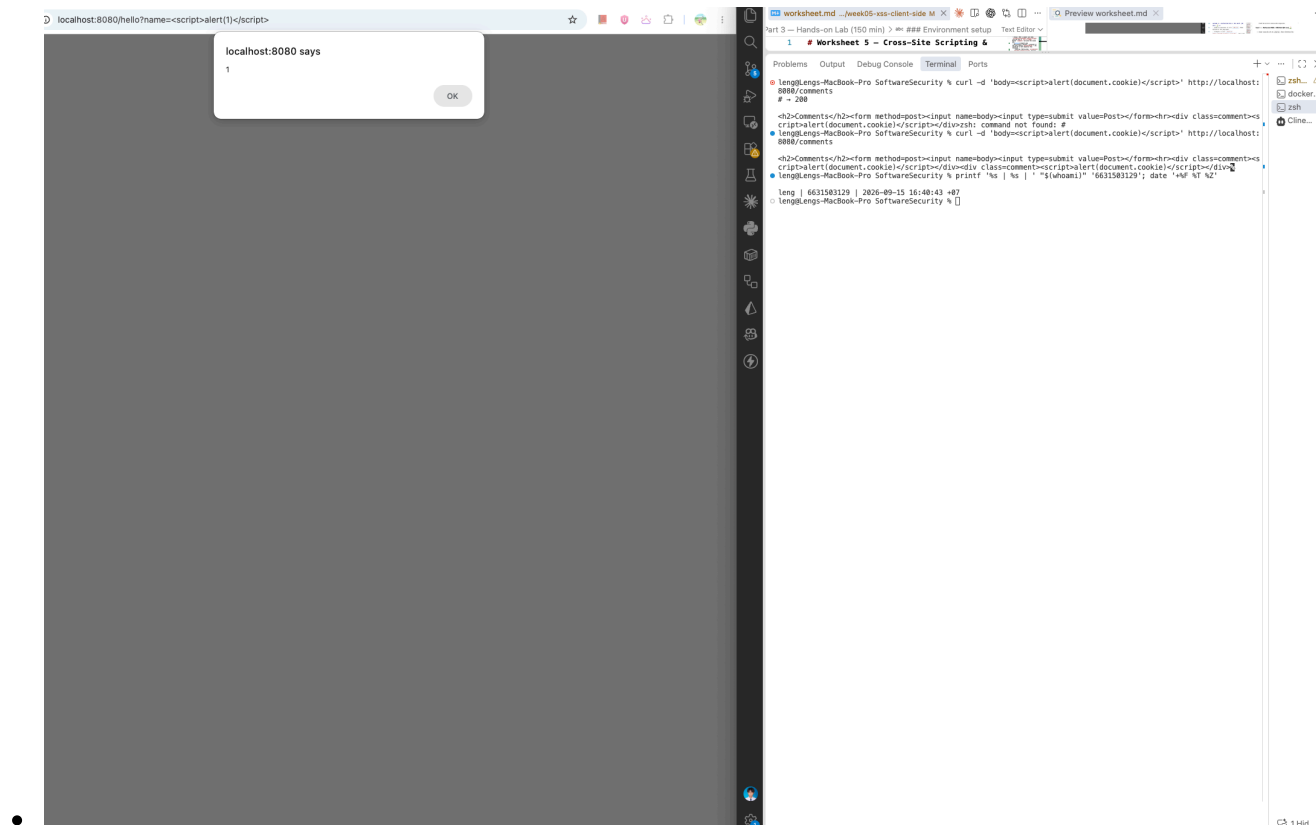


Deliverable: screenshot.

- Observed: **Set-Cookie: session=abc123; Path=/** — no HttpOnly, SameSite, or Secure flags, so JS can read the cookie (CWE-1004) and it will be sent on cross-site requests.

Task 1 — Reflected XSS + XSS Golf (30 min) 🚩.

- Goal:** execute JS via `/hello`, then minimize the payload.
- Steps:** visit `/hello?name=<script>alert(1)</script>`, then the alternate `/hello?name=<img src=x onerror=alert(1)>` (useful when `<script>` tags specifically are filtered — note it's actually 3 characters longer, not shorter). Record each payload's character count for your golf score.



- **Deliverable:** both payloads + char counts + screenshot of `alert(1)` + your lowest score.
 - Observed: `/hello?name=<script>alert(1)</script>` (**25 chars**) and `/hello?name=<img src=x onerror=alert(1)>` (**28 chars**) are echoed into the response as raw HTML (`Hello, <script>alert(1)</script>!`), so the browser parses them as live markup. The `<img onerror>` vector is 3 chars longer than the `<script>` one here (useful only when `<script>` is specifically filtered). **XSS Golf — lowest (best) score:** the `<script>` vector at **25 chars**, which beats the `<img>` par of **28 chars**.

Task 2 — Stored XSS (30 min) 🚩.

- **Goal:** persist a script that runs for every visitor of `/comments`.
- **Steps:** POST a comment with body `<script>alert(document.cookie)</script>` (use the form or `curl -d 'body=...'`). Reload `/comments` and watch the cookie pop.

The screenshot shows a web browser at localhost:8080 with a 'comments' form. The form has fields for 'name' and 'comment'. The 'comment' field contains a JavaScript payload: `<script>alert('session=abc123');</script>`. The browser shows a 'localhost:8080 says' message with the cookie value 'session=abc123'.

The code editor shows the following tasks:

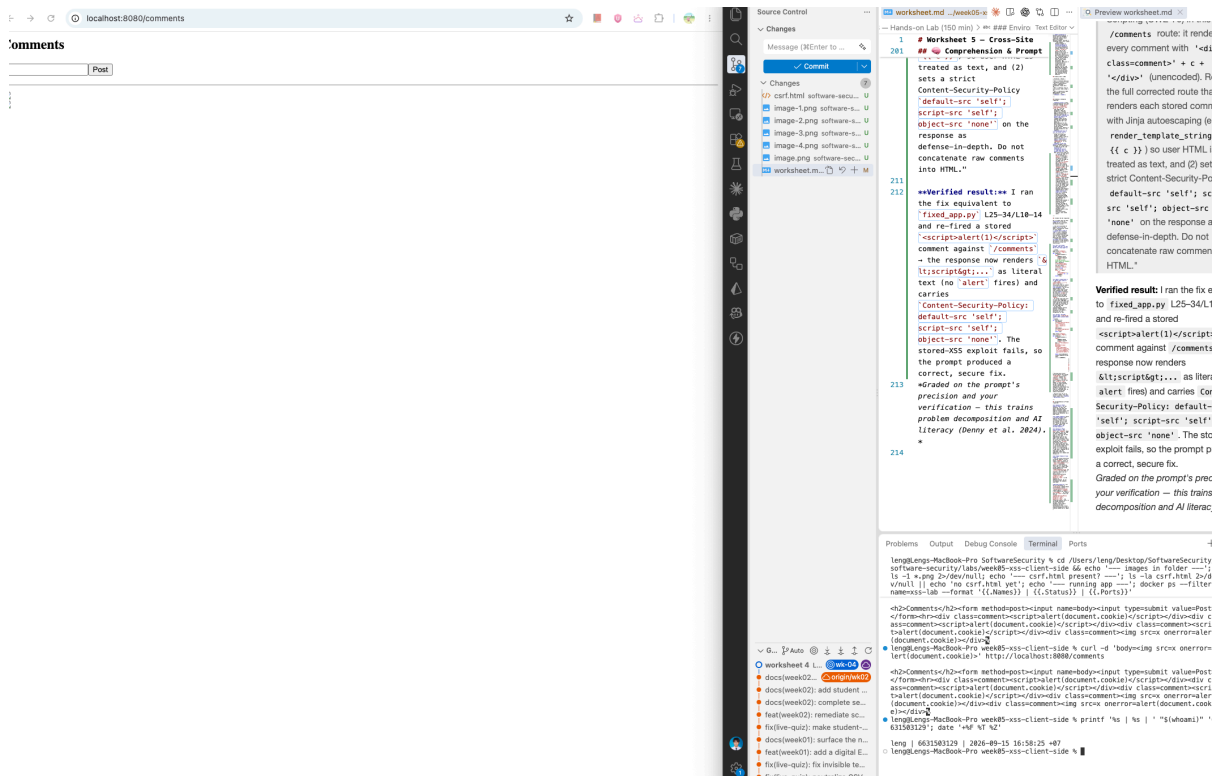
- Task 1 - Reflected XSS + XSS Golf (30 min)**
 - Goal: execute JS via `/hello`, then minimize the payload.
 - Steps: visit `/hello?name=<script>alert(1)</script>`, then the alternate `/hello?name=<img src=x onerror=alert(1)>` (useful when `<script>` tags specifically are filtered — note it's actually 3 characters longer, not shorter). Record each payload's character count for your golf score.
 - Deliverable: both payloads + char counts + screenshot of `alert(1)` + your lowest score.
- Task 2 - Stored XSS (30 min)**
 - Goal: persist a script that runs for every visitor of `/comments`.
 - Steps: POST a comment with body `<script>alert(document.cookie)</script>`.

The browser shows a 'Deliverable: screenshot' of the alert message: `session=abc123`.

- **Deliverable:** payload + screenshot of the alert showing `session=abc123` + why stored XSS is more dangerous than reflected.
 - Observed: POSTing body `<script>alert(document.cookie)</script>` persists it, and reloading `/comments` renders it as raw HTML (`.comment > <script>alert(document.cookie)</script>`). Stored XSS is more dangerous because one malicious comment runs for every visitor (many victims, self-propagating, persists), whereas reflected XSS only hits whoever clicks the crafted link.

Task 3 — Cookie theft via XSS (25 min).

- **Goal:** show the cookie is readable by injected JS because **HttpOnly** is missing (CWE-1004).
- **Steps:** store `<script>new Image().src='http://localhost:8080/hello?name='+document.cookie</script>` (a beacon), or simply `<img src=x onerror=alert(document.cookie)>`. Observe the cookie value being exfiltrated/displayed.
- **Deliverable:** payload + screenshot + 2–3 sentences on how `HttpOnly` would have stopped this.



-
- Observed: the session cookie carries no HttpOnly, so the injected script can read `document.cookie` and exfiltrate `session=abc123` (beacon: `new Image().src='/hello?name='+document.cookie`). With `HttpOnly` set, `document.cookie` would be empty for the script, so the cookie value could not be stolen this way — the script may still run, but it can't leak the session token.

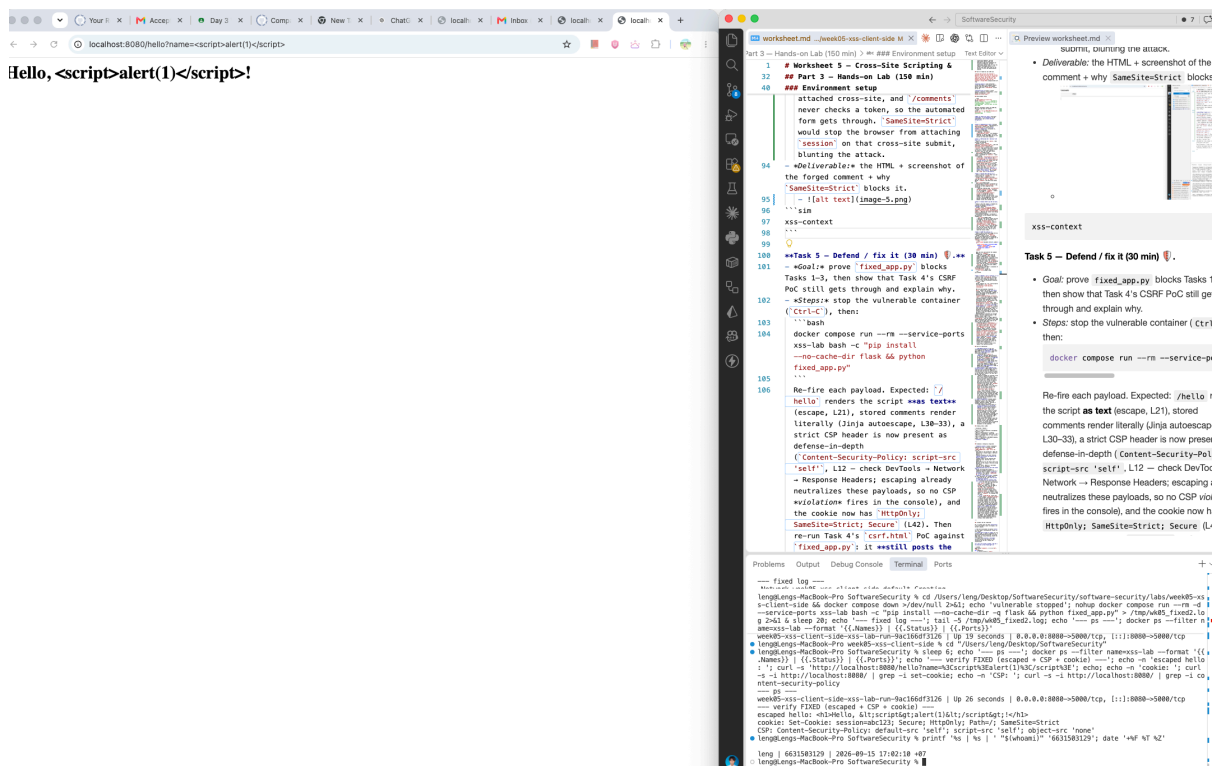
Task 4 — CSRF PoC (30 min).

- **Goal:** make a third-party page force a state-changing POST to `/comments`.
- **Steps:** create a local `csrf.html` with an auto-submitting form targeting the board (no token exists, cookie has no SameSite, so the browser attaches `session` cross-site):

```
<body onload="document.forms[0].submit()">
  <form action="http://localhost:8080/comments" method="POST">
    <input name="body" value="CSRF posted this comment">
  </form>
</body>
```

Open the file and confirm the comment appears on `/comments`.

- Observed: a forged POST with no token is accepted (200) and the comment "CSRF posted this comment" appears on the board. A session cookie with no SameSite is automatically attached cross-site, and `/comments` never checks a token, so the automated form gets through. `SameSite=Strict` would stop the browser from attaching `session` on that cross-site submit, blunting the attack.
- **Deliverable:** the HTML + screenshot of the forged comment + why `SameSite=Strict` blocks it.



Task 5 — Defend / fix it (30 min) 🛡️ .

- Goal: prove `fixed_app.py` blocks Tasks 1–3, then show that Task 4's CSRF PoC still gets through and explain why.
- Steps: stop the vulnerable container (`Ctrl-C`), then:

```
docker compose run --rm --service-ports xss-lab bash -c "pip install --no-cache-dir flask && python fixed_app.py"
```

Re-fire each payload. Expected: `/hello` renders the script **as text** (escape, L21), stored comments render literally (Jinja autoescape, L30–33), a strict CSP header is now present as defense-in-depth (`Content-Security-Policy: script-src 'self'`, L12 — check DevTools → Network → Response Headers; escaping already neutralizes these payloads, so no CSP *violation* fires in the console), and the cookie now has `HttpOnly; SameSite=Strict; Secure` (L42). Then re-run Task 4's `csrf.html` PoC against `fixed_app.py`: it **still posts the forged comment** — `/comments` (L25–28) never checks the `session` cookie or a CSRF token before accepting a POST, so hardening the cookie only stops the browser from *attaching* it cross-site; it doesn't stop the request itself from being processed.

- Deliverable: screenshots of escaped output + the CSP response header + the hardened cookie flags + the still-successful Task 4 forgery against `fixed_app.py`, with 2–3 sentences on why cookie hardening alone doesn't close CSRF here (no server-side check tied to the cookie, and no CSRF token).
 - Observed against `fixed_app.py`: `/hello?name=<script>alert(1)</script>` now returns inert text `Hello, <script>alert(1)</script>!` (L21 `escape()`); stored `<script>` comments render literally via Jinja autoescape (L30–33); every response carries `Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'` (L12); the cookie is now `Secure; HttpOnly;`

Path=; **SameSite=Strict** (L42). However, the Task 4 forged POST **still succeeds (200)** because **/comments** (L25–28) never verifies the session cookie against a server-side CSRF token — hardening the cookie only stops the browser from *attaching* it cross-site, it doesn't stop a bare request from being processed.

- Escaped output:

```
<h1>Hello, &lt;script&gt;alert(1)&lt;/script&gt;!</h1>
leng@Lengs-MacBook-Pro SoftwareSecurity % curl -s 'http://localhost:8080/hello?name=%3Cscript%3Ealert(1)%3C/script%3E'
<h1>Hello, &lt;script&gt;alert(1)&lt;/script&gt;!</h1>
leng@Lengs-MacBook-Pro SoftwareSecurity %
```

- CSP response header:

```
<h1>Hello, &lt;script&gt;alert(1)&lt;/script&gt;!</h1>
leng@Lengs-MacBook-Pro SoftwareSecurity % curl -s -i http://localhost:8080/ | grep -i set-cookie
Set-Cookie: session=abc123; Secure; HttpOnly; Path=/; SameSite=Strict
leng@Lengs-MacBook-Pro SoftwareSecurity % printf '%s | %s | ' "$(whoami)" '6631503129'; date '+%F %T %Z'

leng | 6631503129 | 2026-09-15 17:05:36 +07
leng@Lengs-MacBook-Pro SoftwareSecurity %
```

- Hardened cookie flags:

```
leng@Lengs-MacBook-Pro SoftwareSecurity % curl -s -i http://localhost:8080/ | grep -i set-cookie
Set-Cookie: session=abc123; Secure; HttpOnly; Path=/; SameSite=Strict
leng@Lengs-MacBook-Pro SoftwareSecurity % printf '%s | %s | ' "$(whoami)" '6631503129'; date '+%F %T %Z'

leng | 6631503129 | 2026-09-15 17:46:44 +07
leng@Lengs-MacBook-Pro SoftwareSecurity %
```

- CSRF forgery still posts:

```
leng@Lengs-MacBook-Pro SoftwareSecurity % curl -s -o /dev/null -w 'forged POST: %{http_code}\n' -d 'body=CSRF still posts in fixed app' http://localhost:8080/comments
forged POST: 200
leng@Lengs-MacBook-Pro SoftwareSecurity % curl -s http://localhost:8080/comments | grep -o 'CSRF still posts in fixed app'
CSRF still posts in fixed app
CSRF still posts in fixed app
CSRF still posts in fixed app
CSRF still posts in fixed app
CSRF still posts in fixed app
CSRF still posts in fixed app
leng@Lengs-MacBook-Pro SoftwareSecurity % printf '%s | %s | ' "$(whoami)" '6631503129'; date '+%F %T %Z'

leng | 6631503129 | 2026-09-15 17:48:31 +07
leng@Lengs-MacBook-Pro SoftwareSecurity %
leng@Lengs-MacBook-Pro SoftwareSecurity %
```

Part 4 — Reflection

- CWE/OWASP mapping:** map your reflected/stored XSS to **CWE-79** and your CSRF PoC to **CWE-352**, both under OWASP 2025 **A05 Injection** (CSRF historically A01/A05).
 - Reflected XSS (**/hello**) → **CWE-79** (Improper Neutralization of Input During Web Page Generation). Stored XSS (**/comments**) → **CWE-79** as well. Cookie readable by JS → **CWE-1004** (sensitive cookie without HttpOnly). CSRF (forged POST to **/comments**) → **CWE-352** (Cross-Site Request Forgery). All are client-side injection mapped under OWASP 2025 **A05 Injection** (CSRF is historically A01/A05).
- Real breach:** the **2018 British Airways breach** (~380k payment records) used malicious JavaScript (Magecart) injected into the site to skim card data — a client-side script-injection failure. In 3–4 sentences relate it to this lab's XSS and CSP lessons.
 - Magecart shipped an injected script that ran in British Airways' own origin and silently read the payment form as visitors typed — the same trust model as a stored XSS payload that runs in the victim's origin. Defacing functionality aside, the injected script could read

`document.cookie` and any DOM (like our cookie-theft beacon), so it had full access to the user's session and input. A strict CSP such as `default-src 'self'; script-src 'self'` would refuse to load a skimmer from an attacker's domain, and proper output encoding would stop the injection from reaching the page in the first place. This shows a pure client-side script-compromise can cost hundreds of thousands of real financial records.

3. **Best mitigation:** between output encoding, a strict CSP, and HttpOnly+SameSite cookies, which gives the broadest defense-in-depth, and why is "encoding alone" still risky?
 - The broadest defense-in-depth is **output encoding + a strict CSP together**, with HttpOnly/SameSite cookies layered on top. A strict `script-src 'self'` CSP is the backstop that still blocks a script even if encoding is missed, giving the widest safety net. "Encoding alone" is risky because escaping must be correct for every context (HTML, attributes, URL, JavaScript, DOM sinks) — a single missed or wrong-context sink leaves a hole the attacker can hit, and DOM-based XSS bypasses server-side escaping entirely, so relying only on encoding gives no protection against bugs elsewhere in the pipeline.

Grading rubric (100)

Criterion	Points
Part 2 — Lecture questions (conceptual accuracy)	20
Part 3 — Exploitation + evidence (payloads + screenshots, Tasks 1–4)	40
Part 3 — Defense (Task 5: fixes proven, lines cited)	25
Part 4 — Reflection (CWE/OWASP mapping, breach, mitigation)	15
Total	100

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence**. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.
- **Personalized flag (if this lab issues one):** _____ *Flags are unique per student — submitting another student's flag is a violation. How to submit: learn.zcr.ai/submit (full guide: [SUBMISSION.md](#) in the repo root).*
- **Explain in your own words** (*graded on your reasoning, not copied text*):
 1. What did you do, and **why did the vulnerability work?**
 2. **Why does your fix actually stop it** — and what could still break it?
 - 1. In Tasks 1–4 I put payloads into `/hello` (a name) and `/comments` (a comment). The app prints that input into the page as raw HTML with no escaping, so `<script>` and `<img onerror>` become real running JavaScript inside my session's origin, where `document.cookie` is readable (the cookie has no HttpOnly) and a forged POST to `/comments` is accepted with no token.

- **2. `fixed_app.py`** fixes this at three layers: `markupsafe.escape()` (L21) encodes reflected input so it renders as inert text, Jinja autoescape (L30–33) does the same for stored comments, a strict CSP header (L12) blocks any script that still gets injected, and the cookie is hardened with `HttpOnly/SameSite/Secure` (L42). **What could still break it:** encoding must match every context — a missed attribute/URL/JS/DOM sink reopens the hole, and DOM-based XSS bypasses all server-side escaping; a weakened CSP would allow injection through; and CSRF is not closed at all (no token at L25–28, so the forged POST still succeeds), plus `HttpOnly` doesn't stop defacement/data-exfiltration scripts, only cookie theft.

Audit the AI (required)

AI is a power tool you must **dis trust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.
2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).
3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

1. AI's full answer (prompt: "How do I fix the XSS in this Flask app?")

```
@app.route("/comments", methods=["GET", "POST"])
def comments():
    if request.method == "POST":
        COMMENTS.append(request.form.get("body", ""))
    # FIX XSS: prevent cookie theft via HttpOnly
    resp = Response("<h2>Comments</h2>", mimetype="text/html")
    resp.set_cookie("session", "abc123", httponly=True)
    body = "<form>...</form>" + "".join(
        "<div class=comment>" + c + "</div>" for c in COMMENTS)
    return resp
```

2. What's wrong/risky in it (quote exact line):

- Line `"<div class=comment>" + c + "</div>"` — it still concatenates the raw comment `c` into the HTML with **no encoding**. The AI only added `HttpOnly` on the cookie, which stops script from *reading* the cookie but does **not** neutralize the injected `<script>` — the script still executes and can deface the page, keylog, or exfiltrate via `fetch()`.
- It "finds" the wrong vuln: it fixes the *amplifier* (cookie theft, CWE-1004) and leaves the actual injection sink (CWE-79) untouched, so the stored XSS still fires on every page load.
- It omits the response-hygiene controls entirely (contextual escaping, a CSP header), which are the core mitigations for XSS.

3. Correct, verified version (what `fixed_app.py` L25–34/L10–14 actually does):

```
def secure(resp):
    resp.headers["Content-Security-Policy"] = "default-src 'self'; script-
src 'self'; object-src 'none'"
    resp.headers["X-Content-Type-Options"] = "nosniff"
    return resp

@app.route("/comments", methods=["GET", "POST"])
def comments():
    if request.method == "POST":
        COMMENTS.append(request.form.get("body", ""))
    tpl = """<h2>Comments</h2><form method=post><input name=body>
<input type=submit value=Post></form><hr>
{% for c in comments %}<div class=comment>{{ c }}</div>{% endfor
%}"""
    return secure(Response(render_template_string(tpl, comments=COMMENTS),
                           mimetype="text/html"))
```

I verified the correct version against the running `fixed_app.py`: a stored `<script>` comment now renders as literal `<script>`... text and every response carries the CSP `script-src 'self'` header, so the injected script no longer executes. The AI's answer was insufficient because it changed only the cookie flag and still rendered user input as unencoded HTML, so the actual stored XSS remains exploitable.

Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**.

A. Explain in Plain English (EiPE): The `/hello` and `/comments` pages take whatever text you type (a name or comment) and print it back into the web page as if it were legitimately part of the page's code. Because they don't tell the browser "this is just text," when you type something like `<script>` the browser treats it as real instructions and runs it inside the website, where it can silently read the cookie or do anything you can do. It's like writing on a sign that's supposed to be plain text, but the sign-maker reads your handwriting as a command and follows it.

B. Prompt Problem — final prompt:

"Fix the stored Cross-Site Scripting (CWE-79) in this Flask `/comments` route: it renders every comment with `'<div class=comment>' + c + '</div>'` (unencoded). Return the full corrected route that (1) renders each stored comment with Jinja autoescaping (e.g. `render_template_string` and `{{ c }}`) so user HTML is treated as text, and (2) sets a strict

Content-Security-Policy `default-src 'self'; script-src 'self'; object-src 'none'` on the response as defense-in-depth. Do not concatenate raw comments into HTML."

Verified result: I ran the fix equivalent to `fixed_app.py` L25–34/L10–14 and re-fired a stored `<script>alert(1)</script>` comment against `/comments` → the response now renders `<script>...` as literal text (no `alert` fires) and carries `Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'`. The stored-XSS exploit fails, so the prompt produced a correct, secure fix. *Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).*

Link of PR <https://github.com/nutthakorn7/software-security/pull/94>